Mobile Pilot — Assets Visible, Operational Pages Empty

Symptoms: mobile workflow runs to completion; admin Assets page shows new assets; Site Visits, Operations / Sessions, Dashboard, and Operations Board all show zero. Both ADMIN and QA Manager affected — visibility scope ruled out by the user. Read-only audit of the create paths, list queries, and live dev DB. No code changes.

ROOT CAUSE — Two independent code facts produce the observed pattern:
(a) The mobile pilot workflow never creates an OperationalSession row. The `POST /site-visits` handler calls `resolveCreateOperationalSession()`, which only computes default values for fields stored on the SiteVisit row itself. It writes nothing to `operationalSession`. So the Operations / Sessions page is expected to remain empty under the mobile-only pilot flow.
(b) Assets can be saved without a SiteVisit link. The server's create-asset path treats `createdDuringVisitId` as optional. If the mobile create-visit call ever fails or returns a payload mobile can't interpret as a visit, the AddAsset call can still succeed (asset attaches to the Substation, not the Visit), which produces visible Assets without a visit/inspection/defect chain.
The Operations Board, Dashboard, and Site Visits page query the SiteVisit table (scoped only by tenantId for ADMIN). If those reads return zero in production, the SiteVisit rows do not exist in admin's tenant. The dev DB cross-check confirms the create path produces rows correctly when the call succeeds, so the production divergence is either a create-time failure in mobile or a tenant mismatch.

1. Is mobile creating Site Visit records?

Code: yes. Mobile's `api.createSiteVisit` (`apps/mobile/src/api.ts:351–381`) is the only call site for `POST /site-visits` in the mobile codebase. The server handler `SiteVisitsService.create` (`apps/api/src/site-visits/site-visits.service.ts:375–496`) validates team, substation, MAINHEAD, then calls `prisma.siteVisit.create({...})` at line 434. A successful call writes one `SiteVisit` row with `status: 'ACTIVE'` (default) and `validationStatus: 'PENDING'`.

Live dev DB cross-check. Dev DB currently has 4 SiteVisit rows, all with `status='ACTIVE'`, tenant `1ef638af-…`. The create path is therefore working in principle. If production has zero SiteVisit rows for admin's tenant, the create call either never fired or failed server-side. Common failure modes after Governance G2:

- **MAINHEAD now required at the DTO level** (`create-site-visit.dto.ts:46`: `@IsUUID() mainheadId!: string`). If mobile's `selectedMainheadId` is empty at submit time, the API returns 400 and no visit is created.
- **Team membership.** Non-admin users must be active members of `dto.teamId`; otherwise the handler throws Forbidden (`site-visits.service.ts:393–411`). The mobile UI does not always make this gate visible.
- **Substation resolution.** `resolveCreateSubstation` finds-or-creates a substation; a code/tenant conflict surfaces as 409 Conflict.
- **GPS validation for new-Pencawang check-ins.** (`site-visits.service.ts:1104`) Missing `checkInLatitude/checkInLongitude/checkInAccuracyMeters` for a new pencawang

produces 400.

2. Is mobile creating Operational Session records?

No — and it never has. The mobile pilot workflow does not call `POST /operational-sessions`. The `resolveCreateOperationalSession` helper used by the SiteVisit create handler (`site-visits.service.ts:1032–1060`) computes default values for the operational-session-shaped columns stored on the SiteVisit row itself (`operationMode`, `operationalScope`, `sessionKind`, `fromPencawangId`, `toPencawangId`, `requiresQAQC`, `reportingGroup`). It performs no `operationalSession.create()`.

Net effect: the admin **Operations / Sessions** page lists `OperationalSession` rows. These are produced by a separate module (`apps/api/src/operational-sessions/*`) not exercised by the mobile pilot flow. The dev DB contains 7 such rows because they were created via a different pathway (seed or admin); zero in pilot is expected if no one calls that endpoint.

3. Are assets being saved without a Site Visit?

Yes, this is possible by design. The server `AssetsService.create` (`apps/api/src/assets/assets.service.ts:29–160`) treats `createdDuringVisitId` as optional. If it is omitted, the asset is created with the column null and no `SiteVisitAsset` link row. The asset still appears on the admin Assets page because that page walks substations and lists their assets, independent of any visit.

Live dev DB confirms this is the dominant case for the current pilot dataset: of the 2 Asset rows present, both have `createdDuringVisitId: null` — even though 4 ACTIVE SiteVisits exist and 3 SiteVisitAsset link rows exist. So at minimum, some assets in the dataset bypass the create-time visit linkage. The link rows must have been added subsequently via `linkSiteVisitAsset` (`POST /site-visits/:id/assets`), or by the implicit-link materialisation that runs during visit completion (`materializeImplicitVisitAssetLinks` at `site-visits.service.ts:722`).

Important caveat: the mobile AddAssetScreen passes `createdDuringVisitId: siteVisitId` at `AddAssetScreen.tsx:476`. If `siteVisitId` is undefined at that point (because the visit creation step failed and the navigation arrived at AddAsset by a non-standard route, or because the visitId was lost from the navigation state), the field is omitted from the JSON payload and the server treats it as not provided — producing the no-link case observed in the dev dataset.

4. Are inspections linked to a Site Visit?

By schema, yes. The `Inspection` model carries an `assetId` and is created through the asset → visit chain. In practice, inspections cannot exist without an asset, and the mobile inspection flow drills in from a visit context. If the mobile create-visit call fails, the user cannot reach the inspection form because the VisitDetail screen is the entry-point.

In the dev dataset the inspection count (4) matches the SiteVisit count (4) — consistent with one inspection per visit. If production has zero inspections, that is consistent with zero visits actually being created in the pilot.

5. Are dashboard queries filtering valid records?

No — the dashboard query is permissive for ADMIN. `DashboardService` (`apps/api/src/dashboard/dashboard.service.ts:657–662`) scopes all SiteVisit reads through `accessibleSiteVisitWhere`:

```
private accessibleSiteVisitWhere(user) {
  return {
    tenantId: user.tenantId,
    ...this.siteVisitAccessScope(user),
  };
}

private siteVisitAccessScope(user) {
  if (user.role === 'ADMIN') return {};
  return { team: { members: { some: { userId: user.id, isActive: true } } } };
}
```

- ADMIN user — only `tenantId: user.tenantId` is applied. All SiteVisit rows in admin's tenant are counted.
- Non-admin user (including MANAGER, SUPERVISOR, TECHNICIAN, "QA Manager") — additionally requires the user to be an active member of the visit's team. If the mobile pilot user is NOT a team member of the visit's team, the dashboard counts zero. **This is the most common cause of QA Manager seeing zero**: there is no `QA_VALIDATOR` role with cross-team visibility today.
- ADMIN and non-admin both seeing zero means: **no rows exist in admin's tenant**. The dashboard isn't hiding them — they aren't there.
- The Site Visits list page uses the same `accessScope` (`site-visits.service.ts:2136–2151`), and `buildListWhere` at lines 1812–1839 also gates on `tenantId`. Same conclusion.
- The Operations Board reads from defects/inspections, which are downstream of SiteVisits. Zero visits → zero defects.

6. Root cause

Decomposed by symptom:

Symptom	Cause
Admin Assets page shows new assets	Assets are tied to **Substation**, not to a SiteVisit. The admin Assets page walks substations and lists their assets, independent of any visit context. Assets are persisted even when `createdDuringVisitId` is null.

Symptom	Cause
Admin Site Visits page shows 0	Either (a) no SiteVisit rows exist in admin's tenant — which means the mobile create-visit call did not produce a row (failed validation, returned 4xx, or never fired); or (b) the rows exist but in a different tenant than admin (rare — would require cross-tenant setup). The ADMIN list query has no role filter beyond `tenantId`.
Admin Operations / Sessions page shows 0	**By design.** Mobile never creates OperationalSession rows. This page is expected to remain empty in mobile-only pilots until an admin flow seeds sessions.
Dashboard shows 0 visits	Same query path as Site Visits page. Same conclusion: rows don't exist in admin's tenant.
Operations Board shows 0 defects	Defects are derived from inspections which require an asset link inside a SiteVisit. Zero visits → zero inspections → zero defects. Downstream of (b).
QA Manager sees all operational pages blank	If QA Manager has role MANAGER (no special QA role exists today), the team-membership scope drops every visit they are not a member of. Even if visits do exist, the QA persona would still see zero on those pages because QA is not in the execution team. This is a known governance gap noted in the V8 / V10 review — QA needs a cross-MAINHEAD override that the current code does not provide for non-ADMIN users.

7. Recommended verification probes on production

Run these read-only Prisma probes against the production DB (DATABASE_URL pointed at prod). Each is independent.

```
// 1. Tenant scoping
prisma.user.findUnique({ where: { email: 'ADMIN_EMAIL' }, select: { tenantId: true }})
prisma.siteVisit.groupBy({ by: ['tenantId'], _count: { _all: true } })
// admin.tenantId should equal at least one SiteVisit tenantId

// 2. Recent SiteVisit rows
prisma.siteVisit.findMany({
  orderBy: { createdAt: 'desc' }, take: 10,
  select: { id, tenantId, teamId, mainheadId, status, validationStatus,
    createdByUserId, createdAt, completedAt }
})

// 3. Recent Assets — note createdDuringVisitId
prisma.asset.findMany({
  orderBy: { createdAt: 'desc' }, take: 10,
  select: { id, tenantId, substationId, assetCode,
    createdDuringVisitId, createdAt }
})
// If most/all rows have createdDuringVisitId=null, that confirms the visit chain
// is broken — assets are persisting without their visit linkage.

// 4. Inspections and Defects
prisma.inspection.count()
prisma.defect.count()
```

- **If probe 1 shows mismatched tenants:** the mobile pilot user's tenant differs from the admin's tenant. Fix tenant configuration; rows already created remain in the "wrong" tenant and require a tenant-migration script (or recreation).
- **If probe 2 returns 0 rows:** no visits exist on production. Check the API `access/error` logs around the pilot session for `POST /site-visits` 4xx responses (MAINHEAD validation, team membership, GPS) and the mobile's view of `/users/me/mainheads` (which can be empty if the user has no MAINHEAD access).
- **If probe 3 shows assets with** `createdDuringVisitId=null`: mobile is persisting assets through the substation path while the visit chain is broken. Confirms (a) the symptom (Assets visible) and (b) that no inspection could have been attached to those assets in a visit context.
- **If probe 4 returns 0:** consistent with the visit chain never producing inspections or defects. Operations Board emptiness is expected.

8. Bottom line

Two facts together explain everything observed without invoking a code regression:
1. The mobile pilot workflow never produces OperationalSession rows (it never did).
2. Assets persist independently of SiteVisits because the create-asset endpoint allows the visit linkage to be absent, and the admin Assets page reads via Substation.

If admin sees zero SiteVisits AND zero downstream entities, then no visit was created in admin’s tenant by the pilot run. The most likely production cause is a create-visit failure (Governance G2 MAINHEAD requirement, team membership, GPS, substation conflict) that the mobile UI does not surface clearly enough — followed by asset creation succeeding because that path does not depend on the visit. Run probes 1–4 above against production to confirm which sub-case applies.